SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

CO1 - ASSESSMENT TOOL - 2

Short Answer

Course Code	**CSA3717**	**Course Title**	**Software Testing**
CO Assessed	**CO1**	**Assessment**	**Assessment Tool 2: Short Answer**
Weightage	**50% of CIA**	**Total Marks**	**100**
CO1 Statement	*Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.* **(BL3)**		
Student Name	**M.DATHU**	**Reg. No.**	**192311158**
Section/Batch	**Slot A/2023**	**Date**	**23-10-2026**

Code of Conduct:

I **M.DATHU 192311158** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M.DATHU

1.A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.

Testing as an engineering activity involves a systematic and planned approach to identifying software defects.

- Requirement analysis: Understand the software requirements.
- Test planning: Define scope, objectives, resources, and schedule.
- Test case design: Prepare test cases with inputs and expected results.
- Test execution: Execute test cases and compare actual and expected outputs.
- Defect reporting: Record and communicate identified defects.
- Retesting and regression testing: Verify corrections and ensure existing functions work correctly.

Conclusion: Structured testing improves test coverage, reduces missed defects, and increases software quality and reliability.

2.A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.

A well-defined testing process ensures that banking software works accurately, securely, and reliably.

- Analyze functional and security requirements.
- Prepare a test plan and test cases.
- Perform unit, integration, system, and acceptance testing.
- Test transaction accuracy and error handling.
- Report defects and verify corrections.
- Perform regression testing after modifications.

Benefits: It reduces financial errors, improves security, ensures reliable transactions, and increases customer confidence.

3.During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.

Testing should be performed systematically throughout the Software Development Life Cycle (SDLC).

- Requirement phase: Review requirements for clarity and completeness.
- Design phase: Review architecture and interfaces.
- Coding phase: Perform code reviews and unit testing.
- Integration phase: Verify communication between modules.
- System phase: Test the complete application.
- Acceptance phase: Validate user requirements.
- Maintenance phase: Perform regression testing after changes.

Conclusion: Structured testing identifies defects early, reduces rework, and improves software quality.

4.A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.

- Error: A human mistake during software development.
- Defect: A flaw in the software caused by an error or another problem.
- Failure: Incorrect software behavior during execution.

Relationship: A human error may introduce a defect, and the defect may cause a failure when the affected software is executed.

Conclusion: Understanding these terms helps identify the root cause of problems and improves debugging.

5.A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.

The principle of early testing saves time and cost states that testing should begin as early as possible in the SDLC.

- Review requirements before coding.
- Check system design.
- Perform code reviews and unit testing.
- Conduct integration and system testing.
- Correct defects as soon as they are identified.

Benefits: Early testing reduces correction costs, minimizes rework, prevents defect propagation, and improves software reliability.

6.During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.

Testers should participate in requirement analysis to identify problems before implementation begins.

- Review requirements for completeness and correctness.
- Identify ambiguous and conflicting statements.
- Clarify requirements with stakeholders.
- Prepare test scenarios and acceptance criteria.
- Identify boundary conditions.
- Maintain requirement-to-test-case traceability.

Conclusion: Early tester involvement reduces misunderstandings, prevents defects, and improves software quality.

7.A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.

Defects can originate at different stages of the SDLC.

- Requirements: Incomplete or ambiguous specifications.
- Design: Incorrect architecture or algorithms.
- Coding: Programming and logical errors.
- Integration: Interface and data-exchange problems.
- Testing: Inadequate test coverage.
- Deployment: Incorrect configuration.
- Maintenance: Defects introduced by modifications.

Conclusion: Clear, complete, and testable requirements reduce implementation errors and prevent defects in later stages.

8.A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.

Boundary Value Analysis (BVA) is a black-box testing technique that checks values at and around the minimum and maximum input limits.

Defect classification: A crash caused by boundary inputs is a boundary-related input-handling defect and may also be a functional or robustness defect.

Steps:

- Identify minimum and maximum input values.
- Select values below, at, and above each boundary.
- Execute the test cases.
- Compare actual and expected results.
- Report and correct defects.

Conclusion: BVA detects boundary-related defects and improves input validation and system stability.

9.A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.

A defect repository stores information about previously identified software defects, their causes, severity, and correction status.

- Analyze historical defect reports.
- Identify recurring defects.
- Design test cases for previously affected functions.
- Add important cases to the regression test suite.
- Prioritize modules with frequent or severe defects.
- Update test cases using lessons learned.

Benefits: It improves test coverage, reduces repeated defects, saves test-design time, and supports continuous quality improvement.

10.An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.

A functional defect occurs when software does not perform its specified function correctly.

An ATM withdrawal may fail because of incorrect transaction logic or improper handling of account updates.

Prevention:

- Test successful and unsuccessful withdrawals.
- Verify balance validation and transaction limits.
- Check account updates and transaction records.
- Test error handling and network interruptions.
- Perform integration, retesting, and regression testing.

Conclusion: Systematic functional testing helps identify transaction defects and improves banking system reliability.

11.A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.

Defect clustering is the principle that many software defects are often concentrated in a small number of modules.

Testing priorities:

- Collect historical defect data.
- Identify modules with high defect counts.
- Evaluate defect severity and business risk.
- Prioritize testing of high-risk modules.
- Monitor defect trends across releases.

Benefits: Defect clustering improves resource allocation, increases defect detection efficiency, and reduces unnecessary testing effort.

12.A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.

A performance defect occurs when software fails to meet specified speed, response-time, throughput, or resource-utilization requirements.

Testing methods:

- Load testing: Checks performance under expected workloads.
- Stress testing: Checks behavior beyond normal workload limits.
- Endurance testing: Evaluates performance over an extended period.
- Scalability testing: Checks performance as workload increases.

Conclusion: Correct classification helps identify performance bottlenecks, select suitable testing methods, and improve system responsiveness and stability.

13.A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.

Regression testing ensures that software modifications do not affect existing functionality.

Reuse of historical data:

- Review previous defect reports.
- Identify frequently defective modules.
- Reuse earlier defect-related test cases.
- Add new cases for related conditions.
- Prioritize critical functions.
- Automate repetitive regression tests.
- Analyze results for recurring defects.

Benefits: Historical data improves test coverage, saves time, prevents defect recurrence, and increases testing efficiency.

14.During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.

Integration testing verifies that different software modules communicate and function correctly together.

Role of the tester:

- Participate in early development discussions.
- Review interface specifications and data formats.
- Prepare integration test cases.
- Communicate requirements clearly.
- Report defects with reproduction steps.
- Support root-cause analysis.
- Retest corrections and perform regression testing.

Conclusion: Early collaboration reduces communication gaps, identifies interface defects, and improves integration quality.

15.A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.

Defect prevention aims to eliminate the root causes of defects rather than repeatedly correcting them.

Techniques:

- Conduct root-cause analysis.
- Review requirements and system design.
- Follow coding standards.
- Perform code reviews.
- Use automated testing.
- Analyze historical defect trends.
- Document lessons learned.
- Train developers and testers.
- Improve processes continuously.

Benefits: Defect prevention reduces recurring defects, lowers maintenance costs, improves reliability, minimizes rework, and increases customer satisfaction.

Conclusion: A proactive defect prevention strategy supports long-term software quality improvement.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions	
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	
Organization	20%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	